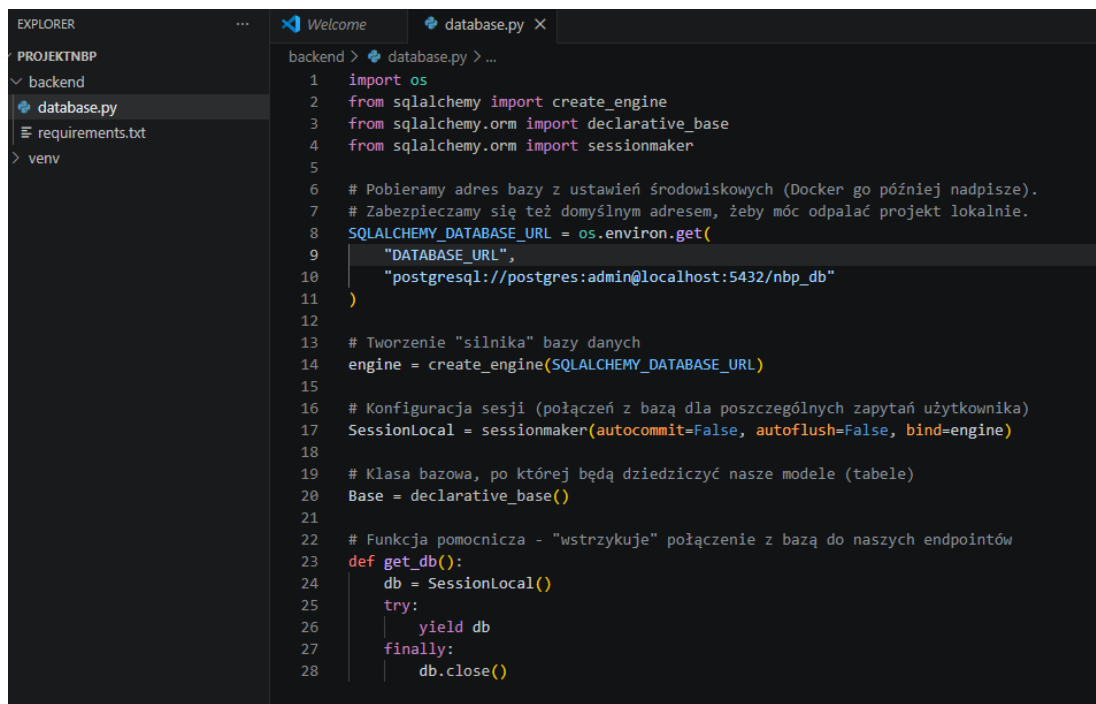
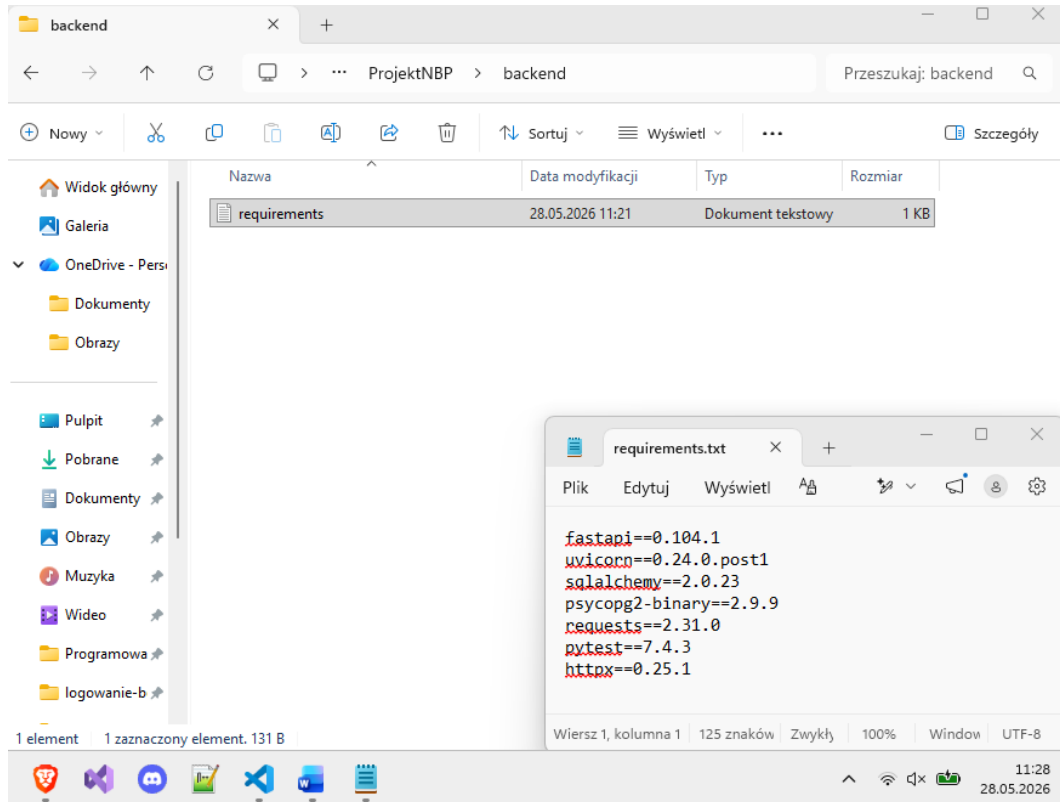


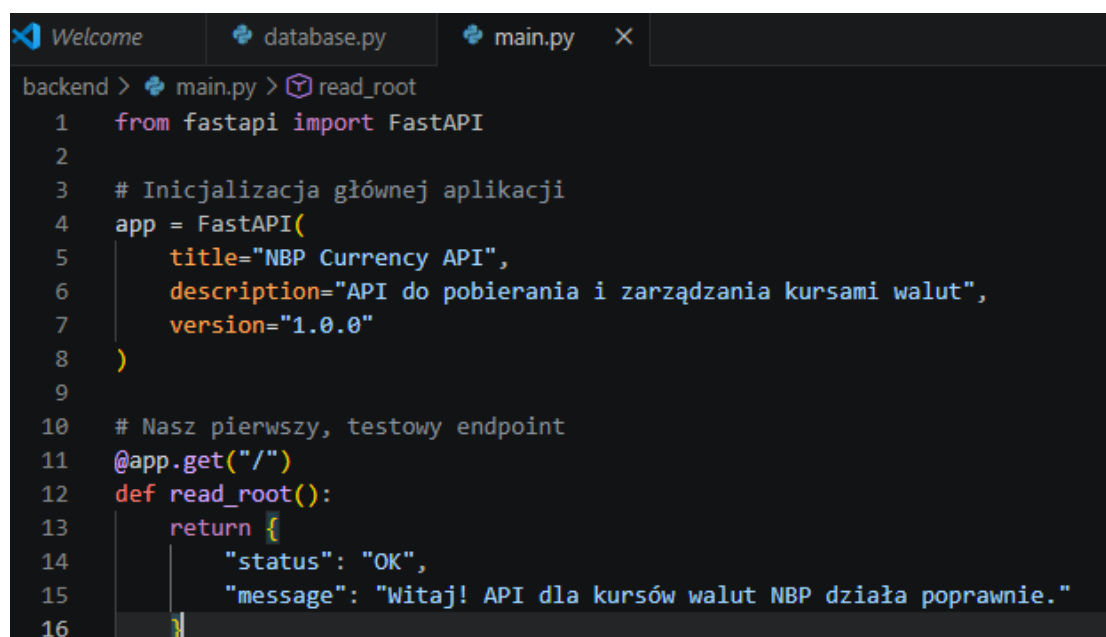
Sprawozdanie z tworzenia projektu zaliczeniowego – ProjektNBP

1. Tworzenie środowiska - Prace nad projektem rozpoczęto od konfiguracji środowiska backendowego. Utworzono plik requirements.txt definiujący wymagane biblioteki (m.in. FastAPI, Uvicorn, SQLAlchemy) i zainstalowano je w środowisku wirtualnym. Utworzono również plik database.py, w którym skonfigurowano silnik lokalnej bazy danych bazujący na narzędziu SQLAlchemy.

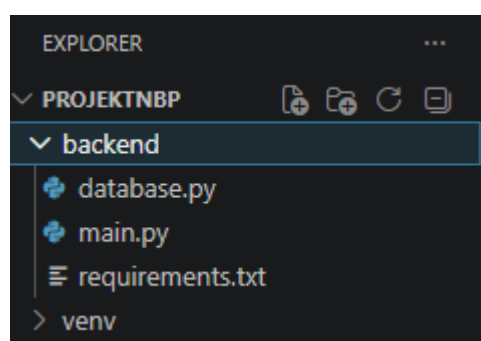


```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\backend> pip install -r requirements.txt
Collecting fastapi==0.104.1 (from -r requirements.txt (line 1))
  Using cached fastapi-0.104.1-py3-none-any.whl.metadata (24 kB)
Collecting uvicorn==0.24.0.post1 (from -r requirements.txt (line 2))
  Using cached uvicorn-0.24.0.post1-py3-none-any.whl.metadata (6.4 kB)
Collecting sqlalchemy==2.0.23 (from -r requirements.txt (line 3))
  Using cached SQLAlchemy-2.0.23-py3-none-any.whl.metadata (9.6 kB)
Collecting psycpg2-binary (from -r requirements.txt (line 4))
  Downloading psycpg2_binary-2.9.12-cp314-cp314-win_amd64.whl.metadata (5.1 kB)
Collecting requests==2.31.0 (from -r requirements.txt (line 5))
  Downloading requests-2.31.0-py3-none-any.whl.metadata (4.6 kB)
Collecting pytest==7.4.3 (from -r requirements.txt (line 6))
```

W głównym pliku main.py zainicjowano obiekt aplikacji FastAPI, nadano jej tytuł i opis, a także stworzono testowy endpoint docelowy (/), który posłużył do weryfikacji, czy serwer uruchamia się i odpowiada poprawnie.

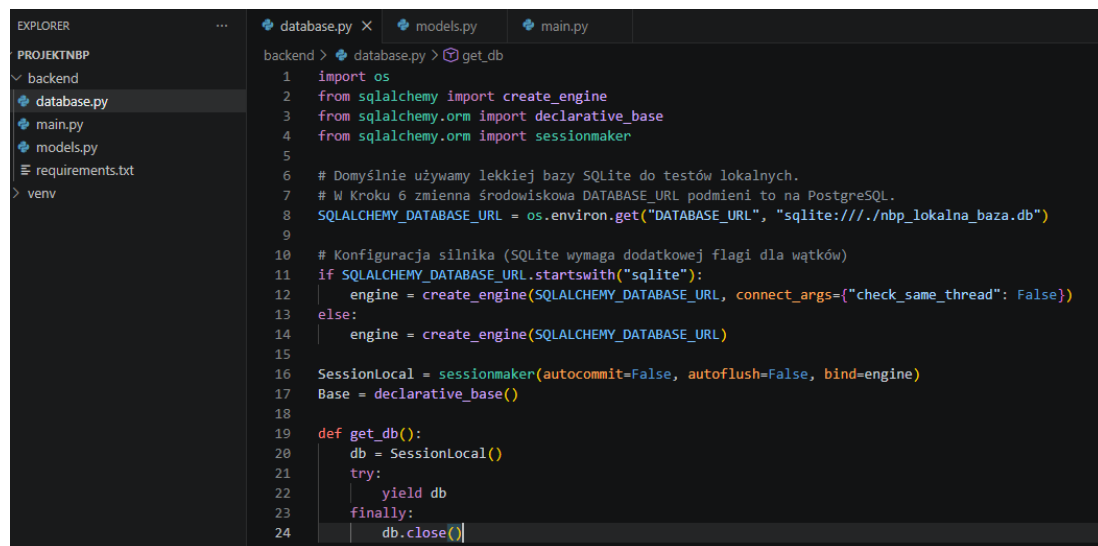


```
backend > main.py > read_root
1  from fastapi import FastAPI
2
3  # Inicjalizacja głównej aplikacji
4  app = FastAPI(
5      title="NBP Currency API",
6      description="API do pobierania i zarządzania kursami walut",
7      version="1.0.0"
8  )
9
10 # Nasz pierwszy, testowy endpoint
11 @app.get("/")
12 def read_root():
13     return {
14         "status": "OK",
15         "message": "Witaj! API dla kursów walut NBP działa poprawnie."
16     }
```



```
EXPLORER
└─ PROJEKTNBP
   └─ backend
      ├── database.py
      ├── main.py
      └── requirements.txt
  > venv
```

2. Zdefiniowano schemat relacyjnej bazy danych dla kursów walut (CurrencyRate) w pliku models.py. Następnie rozbudowano plik main.py o logikę integrującą system z zewnętrznym API Narodowego Banku Polskiego. Stworzono endpoint POST /currencies/fetch, który po wywołaniu odpytuje serwery NBP, przetwarza odpowiedź JSON i zapisuje najnowsze kursy do lokalnej bazy danych. Działanie nowo utworzonych ścieżek zweryfikowano za pomocą wbudowanej w FastAPI dokumentacji interaktywnej Swagger UI.



```
EXPLORER
PROJEKT_NBP
  backend
    database.py
    main.py
    models.py
  requirements.txt
  venv

database.py
1 import os
2 from sqlalchemy import create_engine
3 from sqlalchemy.orm import declarative_base
4 from sqlalchemy.orm import sessionmaker
5
6 # Domyślnie używamy lekkiej bazy SQLite do testów lokalnych.
7 # W Kroku 6 zmienna środowiskowa DATABASE_URL podmieni to na PostgreSQL.
8 SQLALCHEMY_DATABASE_URL = os.environ.get("DATABASE_URL", "sqlite:///./nbp_lokalna_baza.db")
9
10 # Konfiguracja silnika (SQLite wymaga dodatkowej flagi dla wątków)
11 if SQLALCHEMY_DATABASE_URL.startswith("sqlite"):
12     engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False})
13 else:
14     engine = create_engine(SQLALCHEMY_DATABASE_URL)
15
16 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
17 Base = declarative_base()
18
19 def get_db():
20     db = SessionLocal()
21     try:
22         yield db
23     finally:
24         db.close()
```

```
EXPLORER
PROJEKTNBP
  backend
    database.py
    main.py
    models.py
  requirements.txt
  venv

models.py
1 from sqlalchemy import Column, Integer, String, Float, Date
2 from database import Base
3
4 class CurrencyRate(Base):
5     __tablename__ = "currency_rates"
6
7     id = Column(Integer, primary_key=True, index=True)
8     currency = Column(String, index=True) # Pełna nazwa, np. "dolar amerykański"
9     code = Column(String, index=True) # Kod waluty, np. "USD"
10    mid = Column(Float) # Średni kurs, np. 3.95
11    effective_date = Column(Date, index=True) # Data publikacji kursu z NBP
```

```
EXPLORER
PROJEKTNBP
  backend
    database.py
    main.py
    models.py
  requirements.txt
  venv

main.py
1 from fastapi import FastAPI, Depends, HTTPException
2 from sqlalchemy.orm import Session
3 import requests
4 from datetime import date
5
6 import models
7 from database import engine, get_db
8
9 # Ta linijka automatycznie tworzy tabele w bazie podczas startu serwera
10 models.Base.metadata.create_all(bind=engine)
11
12 app = FastAPI(
13     title="NBP Currency API",
14     description="API do pobierania i zarządzania kursami walut",
15     version="1.0.0"
16 )
17
18 @app.post("/currencies/fetch")
19 def fetch_currencies(db: Session = Depends(get_db)):
20     """Pobiera najnowsze kursy z API NBP i zapisuje do bazy danych."""
21
22     # Odpytanie oficjalnego API NBP
23     response = requests.get("http://api.nbp.pl/api/exchangerates/tables/A/?format=json")
24
25     if response.status_code != 200:
26         raise HTTPException(status_code=400, detail="Błąd połączenia z NBP API")
27
28     data = response.json()[0]
29     rates = data['rates']
30     table_date = data['effectiveDate']
31
32     added_count = 0
33
34     for rate in rates:
35         # Zabezpieczenie przed zapisaniem dwa razy tych samych kursów
36         existing = db.query(models.CurrencyRate).filter(
37             models.CurrencyRate.code == rate['code'],
38             models.CurrencyRate.effective_date == table_date
39         ).first()
40
41         if not existing:
42             new_rate = models.CurrencyRate(
43                 currency=rate['currency'],
44                 code=rate['code'],
45                 mid=rate['mid'],
46                 effective_date=table_date
47             )
48             db.add(new_rate)
```

```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\backend> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['E:\\Programowanie zaaw. sem 6\\ProjektNBP\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [9976] using StatReload
INFO: Started server process [3732]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

English news and ea... 127.0.0.1:8000/docs#/default/fetch_currencies_currencies_fetch_post

NBP Currency API 1.0.0 OpenAPI 3.0

/openapi.json

API do pobierania i zarządzania kursami walut

default

POST /currencies/fetch Fetch Currencies

Pobiera najnowsze kursy z API NBP i zapisuje do bazy danych.

Parameters Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header:

Example Value | Schema

"string"

GET /currencies Get All Currencies

GET /currencies/{target_date} Get Currencies By Date

Podczas pierwszych testów integracji z API NBP za pośrednictwem interfejsu Swagger napotkano krytyczny błąd serwera (Internal Server Error 500). Błąd ten wynikał z braku odpowiedniego formatowania i parsowania łańcuchów znaków reprezentujących datę zwracanych przez zewnętrzne API.

POST /currencies/fetch Fetch Currencies

Pobiera najnowsze kursy z API NBP i zapisuje do bazy danych.

Parameters Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'POST' \n'http://127.0.0.1:8000/currencies/fetch' \n-H 'accept: application/json' \n-d ''

Request URL

http://127.0.0.1:8000/currencies/fetch

Server response

Code	Details
500	Error: Internal Server Error

Response body

Internal Server Error

Download

Response headers

content-length: 21\ncontent-type: text/plain; charset=utf-8\ndate: Thu, 28 May 2026 10:15:57 GMT\nserver: unicorn

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Zidentyfikowany problem rozwiązano poprzez zaimplementowanie poprawnej konwersji ciągów znaków (string) na obiekty daty, wykorzystując w tym celu wbudowaną bibliotekę datetime. Wprowadzono również mechanizmy zabezpieczające przed przypadkową duplikacją i nadpisywaniem tych samych kursów walut z tego samego dnia w bazie danych. Po wdrożeniu poprawek, żądanie wykonane poprzez interfejs Swagger UI zakończyło się oczekiwanym statusem 200 OK.

```
database.py  models.py  main.py  X
backend > main.py > fetch_currencies
19 def fetch_currencies(db: Session = Depends(get_db)):
20     response = requests.get("http://api.nbp.pl/api/exchangerates/tables/A/?format=json")
21
22     if response.status_code != 200:
23         raise HTTPException(status_code=400, detail="Błąd połączenia z NBP API")
24
25     data = response.json()[0]
26     rates = data['rates']
27     table_date_str = data['effectiveDate']
28
29     # KRYTYCZNA ZMIANA: Zamiana tekstu na obiekt daty Pythona
30     parsed_date = datetime.strptime(table_date_str, "%Y-%m-%d").date()
31
32     added_count = 0
33
34     for rate in rates:
35         # Zabezpieczenie przed zapisaniem dwa razy tych samych kursów
36         existing = db.query(models.CurrencyRate).filter(
37             models.CurrencyRate.code == rate['code'],
38             models.CurrencyRate.effective_date == parsed_date
39         ).first()
40
41         if not existing:
42             new_rate = models.CurrencyRate(
43                 currency=rate['currency'],
44                 code=rate['code'],
45                 mid=rate['mid'],
46                 effective_date=parsed_date
47             )
48             db.add(new_rate)
49             added_count += 1
50
51     db.commit()
52     return {"message": f"Pomyślnie pobrano i zapisano {added_count} nowych kursów z dnia {table_date_str}."}
53
54 @app.get("/currencies")
55 def get_all_currencies(db: Session = Depends(get_db)):
56     """Zwraca wszystkie kursy walut zapisane w bazie."""
57     return db.query(models.CurrencyRate).all()
58
59 @app.get("/currencies/{target_date}")
60 def get_currencies_by_date(target_date: date, db: Session = Depends(get_db)):
61     """Zwraca kursy walut dla konkretnej daty (format YYYY-MM-DD)."""
62     rates = db.query(models.CurrencyRate).filter(models.CurrencyRate.effective_date == target_date).all()
63
64     if not rates:
65         raise HTTPException(status_code=404, detail="Brak danych dla podanej daty.")
66     return rates
```

POST /currencies/fetch Fetch Currencies

Pobiera najnowsze kursy z API NBP i zapisuje do bazy danych.

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'POST' \n'http://127.0.0.1:8000/currencies/fetch' \n-H 'accept: application/json' \n-d ''

Request URL

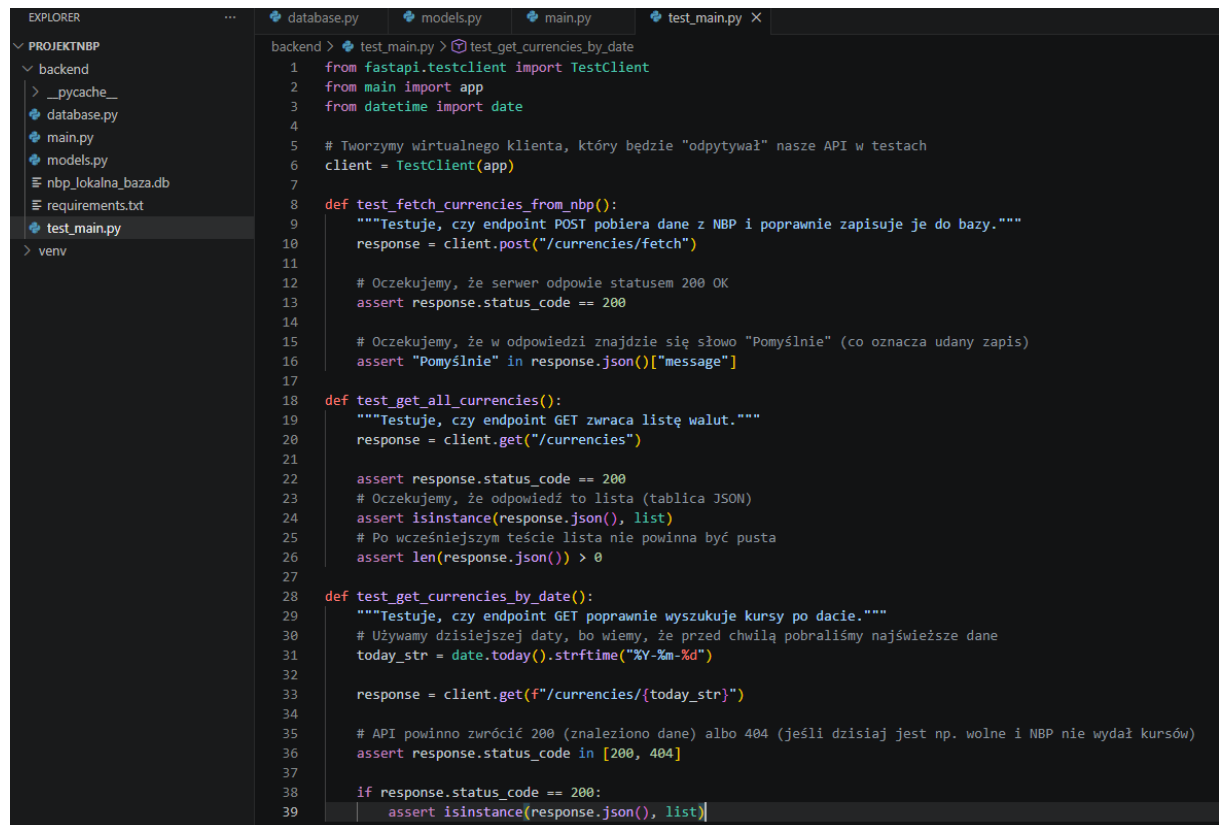
http://127.0.0.1:8000/currencies/fetch

Server response

Code	Details
200	Response body {\n "message": "Pomy\u015blnie pobrano i zapisano 32 nowych kurs\u00f3w z dnia 2026-05-28."\n} Response headers content-length: 89\ncontent-type: application/json\ndate: Thu, 28 May 2026 10:19:35 GMT\nserver: uvicorn

Teraz backend działa bezbłędnie.

3. Aby zapewnić długoterminową stabilność aplikacji, w pliku test_main.py napisano testy automatyczne wykorzystujące środowisko biblioteki pytest. Procedury testowe objęły zarówno endpoint pobierający i transformujący dane z NBP, jak i funkcje zwracające zawartość lokalnej bazy danych. Skrypty testowe przeszły w 100% pomyślnie, ostatecznie potwierdzając bezbłędne, przewidywalne działanie modułu backendowego.

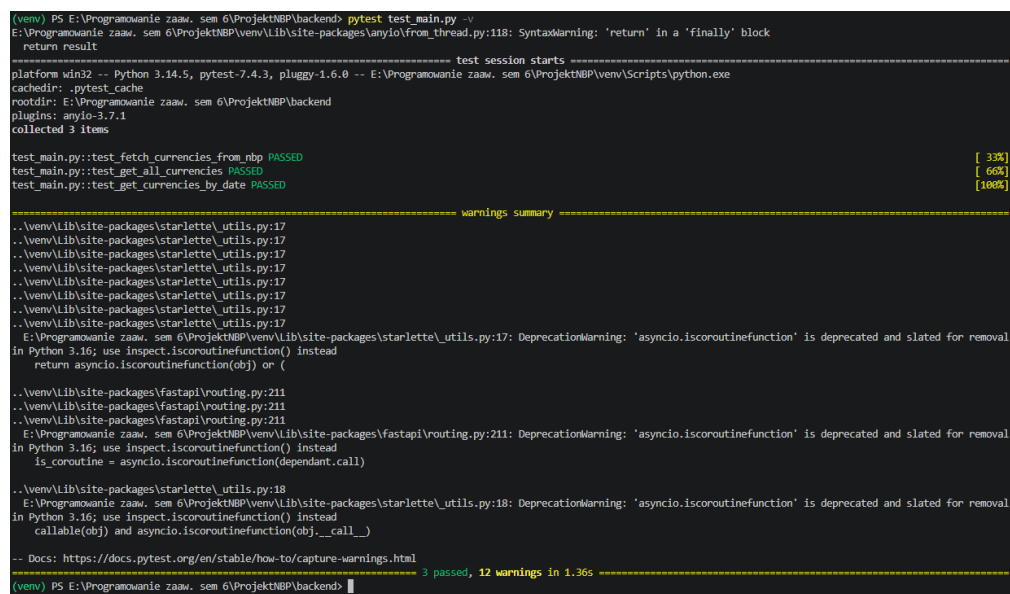


```
EXPLORER
...
database.py
models.py
main.py
test_main.py X

PROJECTNBP
  backend
  _pycache_
  database.py
  main.py
  models.py
  nbp_lokalna_baza.db
  requirements.txt
  test_main.py
  venv

backend > test_main.py > test_get_currencies_by_date
1 from fastapi.testclient import TestClient
2 from main import app
3 from datetime import date
4
5 # Tworzymy wirtualnego klienta, który będzie "odpytywał" nasze API w testach
6 client = TestClient(app)
7
8 def test_fetch_currencies_from_nbp():
9     """Testuje, czy endpoint POST pobiera dane z NBP i poprawnie zapisuje je do bazy."""
10    response = client.post("/currencies/fetch")
11
12    # Oczekujemy, że serwer odpowie statusem 200 OK
13    assert response.status_code == 200
14
15    # Oczekujemy, że w odpowiedzi znajdzie się słowo "Pomyślnie" (co oznacza udany zapis)
16    assert "Pomyślnie" in response.json()["message"]
17
18 def test_get_all_currencies():
19     """Testuje, czy endpoint GET zwraca listę walut."""
20    response = client.get("/currencies")
21
22    assert response.status_code == 200
23    # Oczekujemy, że odpowiedź to lista (tablica JSON)
24    assert isinstance(response.json(), list)
25    # Po wcześniejszym teście lista nie powinna być pusta
26    assert len(response.json()) > 0
27
28 def test_get_currencies_by_date():
29     """Testuje, czy endpoint GET poprawnie wyszukuje kursy po dacie."""
30    # Używamy dzisiejszej daty, bo wiemy, że przed chwilą pobraliśmy najświeższe dane
31    today_str = date.today().strftime("%Y-%m-%d")
32
33    response = client.get(f"/currencies/{today_str}")
34
35    # API powinno zwrócić 200 (znaleziono dane) albo 404 (jeśli dzisiaj jest np. wolne i NBP nie wydał kursów)
36    assert response.status_code in [200, 404]
37
38    if response.status_code == 200:
39        assert isinstance(response.json(), list)
```

Testy przebiegły pomyślnie.



```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\backend> pytest test_main.py -v
E:\Programowanie zaaw. sem 6\ProjektNBP\venv\Lib\site-packages\anyio\from_thread.py:118: SyntaxWarning: 'return' in a 'finally' block
  return result

===== test session starts =====
platform win32 -- Python 3.14.5, pytest-7.4.3, pluggy-1.6.0 -- E:\Programowanie zaaw. sem 6\ProjektNBP\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: E:\Programowanie zaaw. sem 6\ProjektNBP\backend
plugins: anyio-3.7.1
collected 3 items

test_main.py::test_fetch_currencies_from_nbp PASSED [ 33%]
test_main.py::test_get_all_currencies PASSED [ 66%]
test_main.py::test_get_currencies_by_date PASSED [100%]

===== warnings summary =====
..venv\Lib\site-packages\starlette_utils.py:17
..venv\Lib\site-packages\starlette_utils.py:17
..venv\Lib\site-packages\starlette_utils.py:17
..venv\Lib\site-packages\starlette_utils.py:17
..venv\Lib\site-packages\starlette_utils.py:17
..venv\Lib\site-packages\starlette_utils.py:17
E:\Programowanie zaaw. sem 6\ProjektNBP\venv\Lib\site-packages\starlette_utils.py:17: DeprecationWarning: 'asyncio.iscoroutinefunction' is deprecated and slated for removal
in Python 3.16; use inspect.iscoroutinefunction() instead
  return asyncio.iscoroutinefunction(obj) or (
..venv\Lib\site-packages\fastapi\routing.py:211
..venv\Lib\site-packages\fastapi\routing.py:211
..venv\Lib\site-packages\fastapi\routing.py:211
E:\Programowanie zaaw. sem 6\ProjektNBP\venv\Lib\site-packages\fastapi\routing.py:211: DeprecationWarning: 'asyncio.iscoroutinefunction' is deprecated and slated for removal
in Python 3.16; use inspect.iscoroutinefunction() instead
  is_coroutine = asyncio.iscoroutinefunction(dependant.call)
..venv\Lib\site-packages\starlette_utils.py:18
E:\Programowanie zaaw. sem 6\ProjektNBP\venv\Lib\site-packages\starlette_utils.py:18: DeprecationWarning: 'asyncio.iscoroutinefunction' is deprecated and slated for removal
in Python 3.16; use inspect.iscoroutinefunction() instead
  callable(obj) and asyncio.iscoroutinefunction(obj.__call__)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 3 passed, 12 warnings in 1.36s =====
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\backend> █
```


4. Kolejnym krokiem architektonicznym było przygotowanie aplikacji backendowej do bezpiecznej integracji z frontendem. W pliku main.py dodano dedykowany middleware CORS (Cross-Origin Resource Sharing), który znosi restrykcje przeglądarek podczas wymiany danych między różnymi portami. Następnie zainstalowano globalnie narzędzie Angular CLI, wygenerowano czystą strukturę folderów nowej aplikacji front-endowej i pomyślnie uruchomiono jej serwer deweloperski.

```
database.py  models.py  main.py  X  test_main.py
backend > main.py > ...
7   from database import engine, get_db
8
9   # Ta linijka automatycznie tworzy tabele w bazie podczas startu serwera
10  models.Base.metadata.create_all(bind=engine)
11
12  app = FastAPI(
13      title="NBP Currency API",
14      description="API do pobierania i zarządzania kursami walut",
15      version="1.0.0"
16  )
17
18  from fastapi.middleware.cors import CORSMiddleware
19
20  app = FastAPI(
21      title="NBP Currency API",
22      description="API do pobierania i zarządzania kursami walut",
23      version="1.0.0"
24  )
25
26  # KRYTYCZNA ZMIANA: Zezwalamy Angularowi na łączenie się z naszym API
27  app.add_middleware(
28      CORSMiddleware,
29      allow_origins=["*"],
30      allow_credentials=True,
31      allow_methods=["*"],
32      allow_headers=["*"],
33  )
```

```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP> npm install -g @angular/cli
added 288 packages in 37s
70 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New minor version of npm available! 11.13.0 -> 11.16.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.16.0
npm notice To update run: npm install -g npm@11.16.0
npm notice
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP>
```

```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP> cd frontend
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\frontend> ng serve -o
```

Would you like to share pseudonymous usage data about this project with the Angular Team at Google under Google's Privacy Policy at <https://policies.google.com/privacy>. For more details and how to change this setting, see <https://angular.dev/cli/analytics>.

Yes

Thank you for sharing pseudonymous usage data. Should you change your mind, the following command will disable this feature entirely:

```
ng analytics disable
```

Global setting: enabled
Local setting: enabled
Effective status: enabled

Initial chunk files	Names	Raw size
main.js	main	48.48 kB
styles.css	styles	95 bytes
	Initial total	48.58 kB

Application bundle generation complete. [2.070 seconds] - 2026-05-28T11:06:56.557Z

Watch mode enabled. Watching for file changes...

NOTE: Raw file sizes do not reflect development server per-request transformations.

→ Local: <http://localhost:4200/>
→ press **h** + **enter** to show help

```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\frontend> ng generate service services/nbp
CREATE src/app/services/nbp.spec.ts (322 bytes)
CREATE src/app/services/nbp.ts (116 bytes)
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\frontend>
```

Podjęto prace nad warstwą logiczną interfejsu użytkownika. W pliku konfiguracyjnym `app.config.ts` wstrzyknięto narzędzie `HttpClient`, które umożliwia aplikacji webowej komunikację sieciową. Zbudowano od podstaw serwis `nbp.service.ts` definiujący metody uderzające odpowiednio metodami `GET` i `POST` do API stworzonego w języku Python. Ostatecznie, główną logikę widoku, sterującą tabelą z kursami, przypisanodo komponentu sterującego w pliku `app.component.ts`.

```
database.py  models.py  main.py  TS app.config.ts M X  test_main.py
```

```
frontend > src > app > TS app.config.ts > ...
1  import { ApplicationConfig } from '@angular/core';
2  import { provideRouter } from '@angular/router';
3  import { routes } from './app.routes';
4  import { provideHttpClient } from '@angular/common/http';
5
6  export const appConfig: ApplicationConfig = {
7    providers: [
8      provideRouter(routes),
9      provideHttpClient()
10   ]
11 };
```

```

frontend > src > app > services > TS nbp.service.ts > NbpService
1  import { Injectable } from '@angular/core';
2  import { HttpClient } from '@angular/common/http';
3  import { Observable } from 'rxjs';
4
5  @Injectable({ providedIn: 'root' })
6  export class NbpService {
7      private apiUrl = 'http://127.0.0.1:8000/currencies';
8
9      constructor(private http: HttpClient) {}
10
11     fetchFromNbp(): Observable<any> {
12         return this.http.post(`${this.apiUrl}/fetch`, {});
13     }
14
15     getCurrencies(): Observable<any[]> {
16         return this.http.get<any[]>(this.apiUrl);
17     }
18 }

```

```

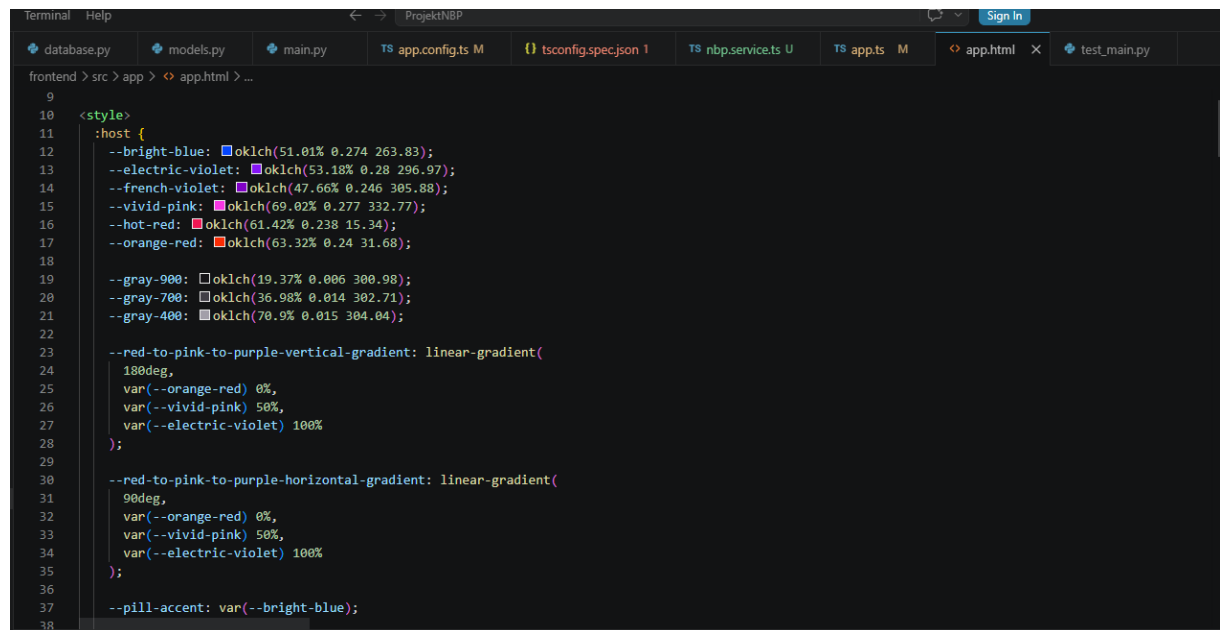
database.py  models.py  main.py  TS app.config.ts M  {} tsconfig.spec.json 1  TS nbp.service.ts U  TS app.ts M X  app.html  test_main.py
frontend > src > app > TS app.ts > AppComponent
1  import { Component, OnInit } from '@angular/core';
2  import { CommonModule } from '@angular/common';
3  import { NbpService } from './services/nbp.service';
4
5  @Component({
6      selector: 'app-root',
7      standalone: true,
8      imports: [CommonModule],
9      templateUrl: './app.component.html',
10     styleUrls: ['./app.component.css']
11 })
12 export class AppComponent implements OnInit {
13     currencies: any[] = [];
14     message: string = '';
15
16     constructor(private nbpService: NbpService) {}
17
18     ngOnInit() {
19         this.loadCurrencies();
20     }
21
22     loadCurrencies() {
23         this.nbpService.getCurrencies().subscribe(data => {
24             this.currencies = data;
25         });
26     }
27
28     fetchNewData() {
29         this.nbpService.fetchFromNbp().subscribe(res => {
30             this.message = res.message;
31             this.loadCurrencies();
32         });
33     }
34 }

```

PS E:\Programowanie zaaw. sem 6\ProjektNBP> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\Programowanie zaaw. sem 6\ProjektNBP\venv\Scripts\Activate.ps1")
 (venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP> ng generate service services/nbp
 Error: This command is not available when running the Angular CLI outside a workspace.
 (venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP> cd frontend
 (venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\frontend> ng generate service services/nbp
 CREATE src/app/services/nbp.spec.ts (322 bytes)
 CREATE src/app/services/nbp.ts (116 bytes)
 (venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\frontend>

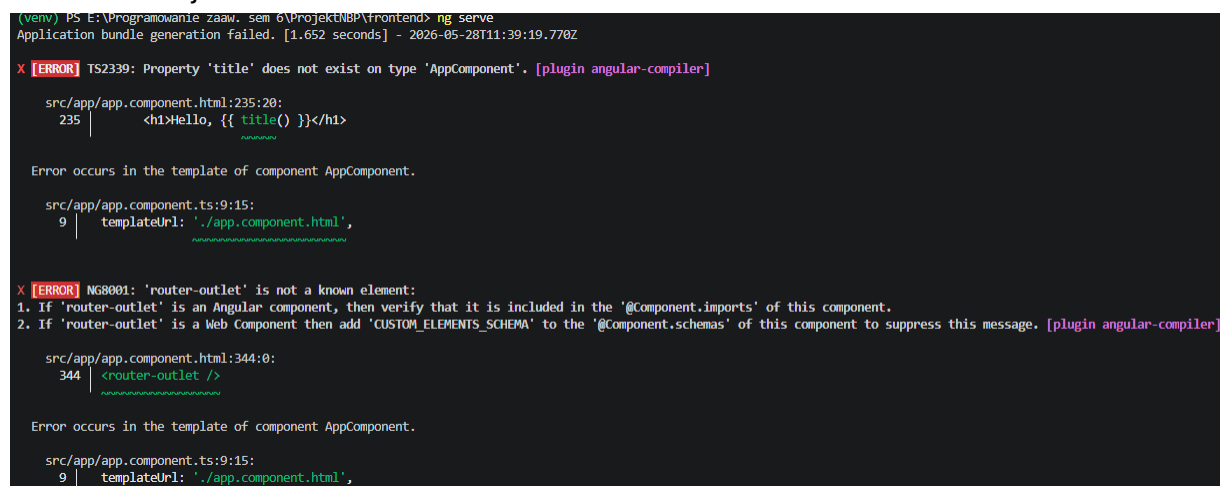
Zdefiniowano podstawową strukturę graficzną i style CSS (np. przyciski i układ tabeli). Podczas pierwszej próby renderingu interfejsu wystąpiły błędy kompilatora na poziomie Angulara, wynikające z pozostałości zmiennych domyślnego szablonu startowego frameworka. Całkowicie oczyszczono plik szablonu app.component.html

i naniesiono nową, własną strukturę HTML aplikacji, co poskutkowało wyświetleniem przejrzystego interfejsu użytkownika.

A screenshot of a code editor window titled 'ProjektNBP'. The editor shows a file named 'app.html' with CSS styles. The styles include a color palette with names like 'bright-blue', 'electric-violet', 'french-violet', 'vivid-pink', 'hot-red', and 'orange-red', each followed by an 'oklch' color function. There are also grayscale styles like 'gray-900', 'gray-700', and 'gray-400'. Two linear gradient styles are defined: 'red-to-pink-to-purple-vertical-gradient' and 'red-to-pink-to-purple-horizontal-gradient'. A 'pill-accent' style is also present, using the 'bright-blue' color. The code is written in a dark-themed editor with syntax highlighting.

```
9
10 <style>
11 :host {
12   --bright-blue: oklch(51.01% 0.274 263.83);
13   --electric-violet: oklch(53.18% 0.28 296.97);
14   --french-violet: oklch(47.66% 0.246 305.88);
15   --vivid-pink: oklch(60.02% 0.277 332.77);
16   --hot-red: oklch(61.42% 0.238 15.34);
17   --orange-red: oklch(63.32% 0.24 31.68);
18
19   --gray-900: oklch(19.37% 0.006 300.98);
20   --gray-700: oklch(36.98% 0.014 302.71);
21   --gray-400: oklch(70.0% 0.015 304.04);
22
23   --red-to-pink-to-purple-vertical-gradient: linear-gradient(
24     180deg,
25     var(--orange-red) 0%,
26     var(--vivid-pink) 50%,
27     var(--electric-violet) 100%
28   );
29
30   --red-to-pink-to-purple-horizontal-gradient: linear-gradient(
31     90deg,
32     var(--orange-red) 0%,
33     var(--vivid-pink) 50%,
34     var(--electric-violet) 100%
35   );
36
37   --pill-accent: var(--bright-blue);
38 }
```

Przeprowadzono pełny test integracyjny architektury Full-Stack. Po upewnieniu się, że serwer Uvicorn nasłuchuje zdarzeń, w interfejsie front-endowym użyto przycisku asynchronicznie odpytującego własne API. Aplikacja pobrała świeży zestaw danych z Narodowego Banku Polskiego, wyeksportowała go do relacyjnej tabeli, a następnie natychmiast zaprezentowała go użytkownikowi za pomocą dynamicznie renderowanej tabeli HTML.

A screenshot of a terminal window showing the output of an Angular build process. The terminal displays two error messages. The first error is 'TS2339: Property 'title' does not exist on type 'AppComponent'. [plugin angular-compiler]' and points to a line in 'src/app/app.component.html' where a 'title()' function is used in an HTML template. The second error is 'NG8001: 'router-outlet' is not a known element:' and points to a line in 'src/app/app.component.html' where a '<router-outlet />' is used. The terminal also shows the start of the build process with 'ng serve' and 'Application bundle generation failed.'.

```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\frontend> ng serve
Application bundle generation failed. [1.652 seconds] - 2026-05-28T11:39:19.770Z

X [ERROR] TS2339: Property 'title' does not exist on type 'AppComponent'. [plugin angular-compiler]

  src/app/app.component.html:235:20:
  235 |         <h1>Hello, {{ title() }}</h1>
      |                               ~~~~~
Error occurs in the template of component AppComponent.

  src/app/app.component.ts:9:15:
  9 |     templateUrl: './app.component.html',
    |     ~~~~~
X [ERROR] NG8001: 'router-outlet' is not a known element:
1. If 'router-outlet' is an Angular component, then verify that it is included in the '@Component.imports' of this component.
2. If 'router-outlet' is a Web Component then add 'CUSTOM_ELEMENTS_SCHEMA' to the '@Component.schemas' of this component to suppress this message. [plugin angular-compiler]

  src/app/app.component.html:344:0:
  344 | <router-outlet />
      | ~~~~~
Error occurs in the template of component AppComponent.

  src/app/app.component.ts:9:15:
  9 |     templateUrl: './app.component.html',
    |     ~~~~~
```



Panel Kursów Walut NBP

Pobierz najnowsze dane z NBP

Brak danych w bazie. Kliknij przycisk powyżej, aby pobrać kursy.

```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP> cd backend
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\backend> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['E:\\Programowanie zaaw. sem 6\\ProjektNBP\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [5084] using StatReload
INFO: Started server process [548]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:55270 - "GET /currencies HTTP/1.1" 200 OK
INFO: 127.0.0.1:55270 - "OPTIONS /currencies/fetch HTTP/1.1" 200 OK
INFO: 127.0.0.1:55270 - "POST /currencies/fetch HTTP/1.1" 200 OK
INFO: 127.0.0.1:55270 - "GET /currencies HTTP/1.1" 200 OK
```



Panel Kursów Walut NBP

Pobierz najnowsze dane z NBP

Waluta	Kod	Kurs sredni	Data publikacji
bat (Tajlandia)	THB	0.1116 PLN	2026-05-28
dolar amerykański	USD	3.6463 PLN	2026-05-28
dolar australijski	AUD	2.5953 PLN	2026-05-28
dolar Hongkongu	HKD	0.4655 PLN	2026-05-28
dolar kanadyjski	CAD	2.6311 PLN	2026-05-28
dolar nowozelandzki	NZD	2.1467 PLN	2026-05-28
dolar singapurski	SGD	2.8503 PLN	2026-05-28
euro	EUR	4.2355 PLN	2026-05-28
forint (Węgry)	HUF	0.01192 PLN	2026-05-28
frank szwajcarski	CHF	4.6224 PLN	2026-05-28
funt szterling	GBP	4.8866 PLN	2026-05-28
hrywna (Ukraina)	UAH	0.0824 PLN	2026-05-28
jen (Japonia)	JPY	0.022867 PLN	2026-05-28
korona czeska	CZK	0.1744 PLN	2026-05-28
korona duńska	DKK	0.5668 PLN	2026-05-28
korona islandzka	ISK	0.029578 PLN	2026-05-28
korona norweska	NOK	0.3928 PLN	2026-05-28
korona szwedzka	SEK	0.3917 PLN	2026-05-28
lej rumuński	RON	0.8072 PLN	2026-05-28
lira turecka	TRY	0.0794 PLN	2026-05-28

5. Usunięto wadliwe i porzucone pliki testowe wynikające z usuwania domyślnych komponentów, a następnie uruchomiono zaktualizowane testy automatyczne frontendu dla modułu HTTP i serwisu nbp.service. Silnik testowy nie wykrył żadnych anomalii, potwierdzając rzetelność napisanej części wizualnej.

```
(venv) PS E:\Programowanie zaaw. sem 6\ProjektNBP\frontend> ng test --watch=false
```

Initial chunk files	Names	Raw size
chunk-CGZBV6BD.js	-	4.07 kB
spec-app-services-nbp.js	spec-app-services-nbp	1.52 kB
init-testbed.js	init-testbed	1.21 kB
vitest-mock-patch.js	vitest-mock-patch	969 bytes
styles.css	styles	56 bytes
Initial total		7.82 kB

```
Application bundle generation complete. [1.303 seconds] - 2026-05-28T11:51:16.847Z
```

```
RUN v4.1.7 E:/Programowanie zaaw. sem 6/ProjektNBP/frontend
```

```
✓ frontend src/app/services/nbp.spec.ts (1 test) 17ms
  ✓ NbpService (1)
    ✓ powinien zostac poprawnie utworzony 16ms
```

```
Test Files 1 passed (1)
  Tests 1 passed (1)
Start at 13:51:17
Duration 1.82s (transform 101ms, setup 399ms, import 37ms, tests 17ms, environment 1.14s)
```

6. Finalnym etapem prac nad projektem była zaawansowana konteneryzacja całego zestawu aplikacji. Przygotowano specyfikację w postaci dwóch oddzielnych plików Dockerfile: jeden budujący i izolujący środowisko dla frameworka FastAPI, bazujące na oficjalnym obrazie Pythona, oraz drugi dystrybuujący środowisko Node.js i serwujący skompilowaną aplikację Angular.

W głównym katalogu całego projektu utworzono plik orkiestratora docker-compose.yml, który bezkolizyjnie połączył w jedną sieć wirtualną oba stworzone środowiska. Ponadto, na tym etapie wyparto testową, plikową bazę danych SQLite na rzecz profesjonalnego, relacyjnego kontenera systemowego PostgreSQL.

```
main.py TS app.config.ts M {} tsconfig.spec.json 1 TS nbp.spec.ts U Dockerfile.txt X T
backend > Dockerfile.txt
1 # Używamy oficjalnego, stabilnego obrazu Pythona
2 FROM python:3.12
3
4 # Ustawiamy katalog roboczy wewnątrz kontenera
5 WORKDIR /app
6
7 # Kopiujemy listę paczek i je instalujemy
8 COPY requirements.txt .
9 RUN pip install --no-cache-dir -r requirements.txt
10
11 # Kopiujemy resztę naszego kodu (main.py, models.py, database.py)
12 COPY . .
13
14 # Wystawiamy port i uruchamiamy serwer
15 EXPOSE 8000
16 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
py main.py TS app.config.ts M {} tsconfig.spec.json 1 Dockerfile.txt U X
frontend > Dockerfile.txt
1 # Używamy środowiska Node.js do uruchomienia Angulara
2 FROM node:20
3
4 WORKDIR /app
5
6 # Najpierw instalujemy zależności (żeby Docker mógł to skeszkować)
7 COPY package*.json ./
8 RUN npm install
9 RUN npm install -g @angular/cli
10
11 # Kopiujemy resztę projektu frontendu
12 COPY . .
13
14 # Uruchamiamy serwer developerski z dostępem z zewnątrz
15 EXPOSE 4200
16 CMD ["ng", "serve", "--host", "0.0.0.0", "--disable-host-check"]
```

EXPLORER

py

main.py

TS app.config.ts M

tsconfig.spec.json 1

Dockerfile.txt U

docker-compose.yml X

PROJEKTNPB

backend

> _pycache_

> .pytest_cache

database.py

Dockerfile.txt

main.py

models.py

nbp_lokalna_baza.db

requirements.txt

test_main.py

frontend

> .angular

> .vscode

> node_modules

> public

> src

.editorconfig

.gitignore

.prettierrc

angular.json M

Dockerfile.txt U

package-lock.json

package.json

README.md

tsconfig.app.json M

tsconfig.json

tsconfig.spec.json 1

> venv

docker-compose.yml

docker-compose.yml

1 version: '3.8'

2

3 services:

4 db:

5 image: postgres:15

6 container_name: nbp_postgres

7 environment:

8 POSTGRES_USER: nbp_user

9 POSTGRES_PASSWORD: nbp_password

10 POSTGRES_DB: nbp_baza

11 ports:

12 - "5432:5432"

13 volumes:

14 - postgres_data:/var/lib/postgresql/data

15

16 backend:

17 build: ./backend

18 container_name: nbp_fastapi

19 ports:

20 - "8000:8000"

21 environment:

22 # Tutaj dzieje się magia! Nadpisujemy zmienną z database.py

23 # Backend automatycznie przełączy się z SQLite na tego PostgreSQL!

24 - DATABASE_URL=postgresql://nbp_user:nbp_password@db:5432/nbp_baza

25 depends_on:

26 - db

27

28 frontend:

29 build: ./frontend

30 container_name: nbp_angular

31 ports:

32 - "4200:4200"

33 depends_on:

34 - backend

35

36 # Zapisujemy dane z bazy na dysku, żeby nie zniknęły po restarcie kontenera

37 volumes:

38 postgres_data:

Skorygowano problemy nazewnictwa i systemowego maskowania rozszerzeń dla plików Dockerfile. Aplikację ożywiono za pomocą docelowej komendy docker compose up --build, która zainicjowała automatyczne pobieranie pakietów operacyjnych, izolację środowisk oraz uruchomienie niezależnych serwerów w wirtualnych kontenerach, o czym upewniono się za pomocą wglądu w narzędzie Docker Desktop. Skonteneryzowany, ostateczny projekt został pozytywnie zweryfikowany z poziomu przeglądarki.

```
PS E:\Programowanie zaaw. sem 6\ProjektNBP> docker compose up --build
time="2026-05-28T14:23:03+02:00" level=warning msg="E:\Programowanie zaaw. sem 6\ProjektNBP\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 10/16
- Image postgres:15 [███ ██████████] Pulling
```

8.8s

The screenshot shows the Docker Desktop application window. The top bar is blue with the Docker logo, 'PERSONAL' label, a search bar, and navigation icons. The left sidebar contains a list of resources: Gordon, Containers (selected), Images, Logs, Volumes, Kubernetes, Builds, Models, Docker Hub, and MCP Toolkit (BETA). The main area is titled 'Containers' and shows 'Container CPU usage' at 0.28% / 800% (8 CPUs available) and 'Container memory usage' at 142.25MB / 3.63GB. A table lists the running container 'projektnbp' with its ID, image, ports, and status. Below the table, there are 'Walkthroughs' for 'Multi-container applications' (8 mins) and 'Containerize your application' (3 mins). At the bottom, a terminal window shows the output of the 'docker compose up' command, including logs for 'nbp_postgres' and 'nbp_fastapi'.

Containers Give feedback

Container CPU usage 0.28% / 800% (8 CPUs available) Container memory usage 142.25MB / 3.63GB Show charts

Search Only show running containers

Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
projektnbp	-	-	-	0.28%	32 s	[Icons]

Walkthroughs

- Multi-container applications 8 mins
- Containerize your application 3 mins

View more in the Learning center

Engine running RAM 3.56 GB CPU 0.13% Disk: 7.40 GB used (limit 1006.85 GB) v4.75.0

```
nbp_postgres | 2026-05-28 12:35:15.311 UTC [64] LOG: database system was shut down at 2026-05-28 12:35:15 UTC
nbp_postgres | 2026-05-28 12:35:15.323 UTC [1] LOG: database system is ready to accept connections
nbp_fastapi | INFO: Started server process [1]
nbp_fastapi | INFO: Waiting for application startup.
nbp_fastapi | INFO: Application startup complete.
nbp_fastapi | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

View in Docker Desktop View Config Enable Watch Detach

Ln 16, Col 65 Spaces: 4

Panel Kursów Walut NBP

Pobierz najnowsze dane z NBP

✓ Pomyślnie pobrano i zapisano 32 nowych kursów z dnia 2026-05-28.

Waluta	Kod	Kurs średni	Data publikacji
bat (Tajlandia)	THB	0.1116 PLN	2026-05-28
dolar amerykański	USD	3.6463 PLN	2026-05-28
dolar australijski	AUD	2.5953 PLN	2026-05-28
dolar Hongkongu	HKD	0.4655 PLN	2026-05-28
dolar kanadyjski	CAD	2.6311 PLN	2026-05-28
dolar nowozelandzki	NZD	2.1467 PLN	2026-05-28
dolar singapurski	SGD	2.8503 PLN	2026-05-28
euro	EUR	4.2355 PLN	2026-05-28
forint (Węgry)	HUF	0.01192 PLN	2026-05-28
frank szwajcarski	CHF	4.6224 PLN	2026-05-28
funt szterling	GBP	4.8866 PLN	2026-05-28
hrywna (Ukraina)	UAH	0.0824 PLN	2026-05-28
jen (Japonia)	JPY	0.022867 PLN	2026-05-28
korona czeska	CZK	0.1744 PLN	2026-05-28
korona duńska	DKK	0.5668 PLN	2026-05-28
korona islandzka	ISK	0.029578 PLN	2026-05-28
korona norweska	NOK	0.3928 PLN	2026-05-28
korona szwedzka	SEK	0.3917 PLN	2026-05-28
lej rumuński	RON	0.8072 PLN	2026-05-28

The screenshot shows the Docker Desktop application window. The top navigation bar includes the Docker logo, "docker.desktop", a "PERSONAL" badge, a search bar, a "Ctrl+K" shortcut button, and icons for help, notifications, Docker Hub, settings, and a menu. On the left sidebar, there are links to Gordon, Containers (selected), Images, Logs, Volumes, Kubernetes, Builds, Models, Docker Hub, MCP Toolkit (marked BETA), and Extensions (disabled).

The main area displays "Containers" with a link to "Give feedback". It shows two summary metrics: "Container CPU usage" at 0.46% / 800% (8 CPUs available) and "Container memory usage" at 587.75MB / 3.63GB. A "Show charts" link is present.

Below the summary is a search bar and a toggle switch labeled "Only show running containers". A table lists four running containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	projektnbp	-	-	-	0.46%	2 minutes ago	[Stop] [Refresh] [More]
☐	nbp_postgres	0e88c039f8da	postgres:15	5432:5432	0.05%	2 minutes ago	[Stop] [Refresh] [More]
☐	nbp_fastapi	33266ed4115e	projektnbp-back	8000:8000	0.25%	2 minutes ago	[Stop] [Refresh] [More]
☐	nbp_angular	3d288069ddc2	projektnbp-front	4200:4200	0.16%	2 minutes ago	[Stop] [Refresh] [More]

At the bottom right, it says "Showing 4 items".

7. Dla przejrzystości, skompletowano plik README.md pełniący funkcję technicznej instrukcji obsługi platformy. Zawiera on kroki niezbędne do jednoklikowego zbudowania, uruchomienia i przetestowania kompletnego, wielomodułowego oprogramowania za pośrednictwem rozwiązania wirtualizacji Docker.

```
① README.md X
① README.md
1 Projekt NBP - Aplikacja Full-Stack (Angular + FastAPI + PostgreSQL)
2
3 Jak uruchomić projekt:
4
5 Upewnij się, że masz zainstalowanego i uruchomionego Dockera (Docker Desktop).
6
7 Otwórz terminal w głównym folderze projektu (tam, gdzie znajduje się plik docker-compose.yml).
8
9 Wpisz komendę: docker compose up --build
10
11 Poczekaj na zbudowanie kontenerów i uruchomienie serwerów.
12
13 Aplikacja frontendowa będzie dostępna w przeglądarce pod adresem: http://localhost:4200
14
15 Dokumentacja API (Swagger) jest dostępna pod adresem: http://localhost:8000/docs
16
17 Uwaga: Przy pierwszym uruchomieniu tabela walut jest pusta. Należy kliknąć przycisk pobierania na stronie, aby zainicjować rzut danych z API NBP do nowej bazy PostgreSQL.
```